

Praetor — Development Conventions

How we build: structure, naming, coding, Git, Docker

Version 1.0.0

Table of Contents

1. Purpose & scope

This doc is the **single reference for how we build Praetor day to day** — file structure, naming, coding practices, Git, and Docker. It exists so three people write code that looks like it came from one, and so anyone can drop into anyone else's module without relearning the rules.

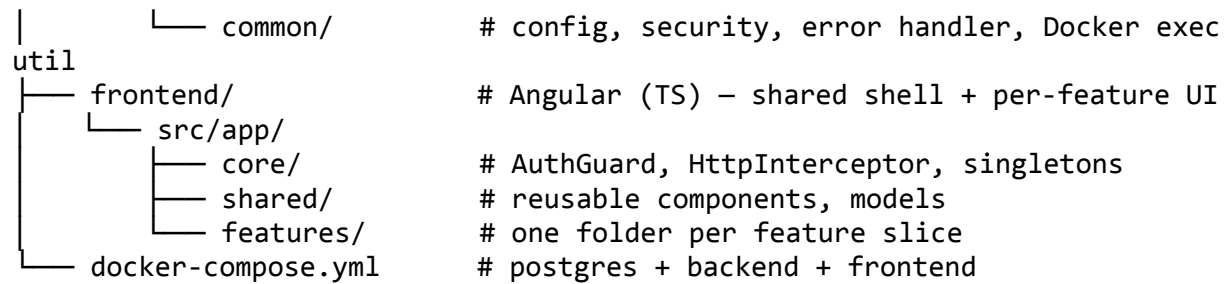
It complements, does not replace:

- `docs/praetor-team-plan.md` — process, sprints, module ownership, Definition of Done.
- `docs/api-contracts.md` — the exact endpoint shapes. **Implement them, don't redesign them.**
- `db/schema.sql` — the tables. Source of truth for the data model.

If anything here ever conflicts with those three files, **they win** — fix this doc.

2. Repo & file structure

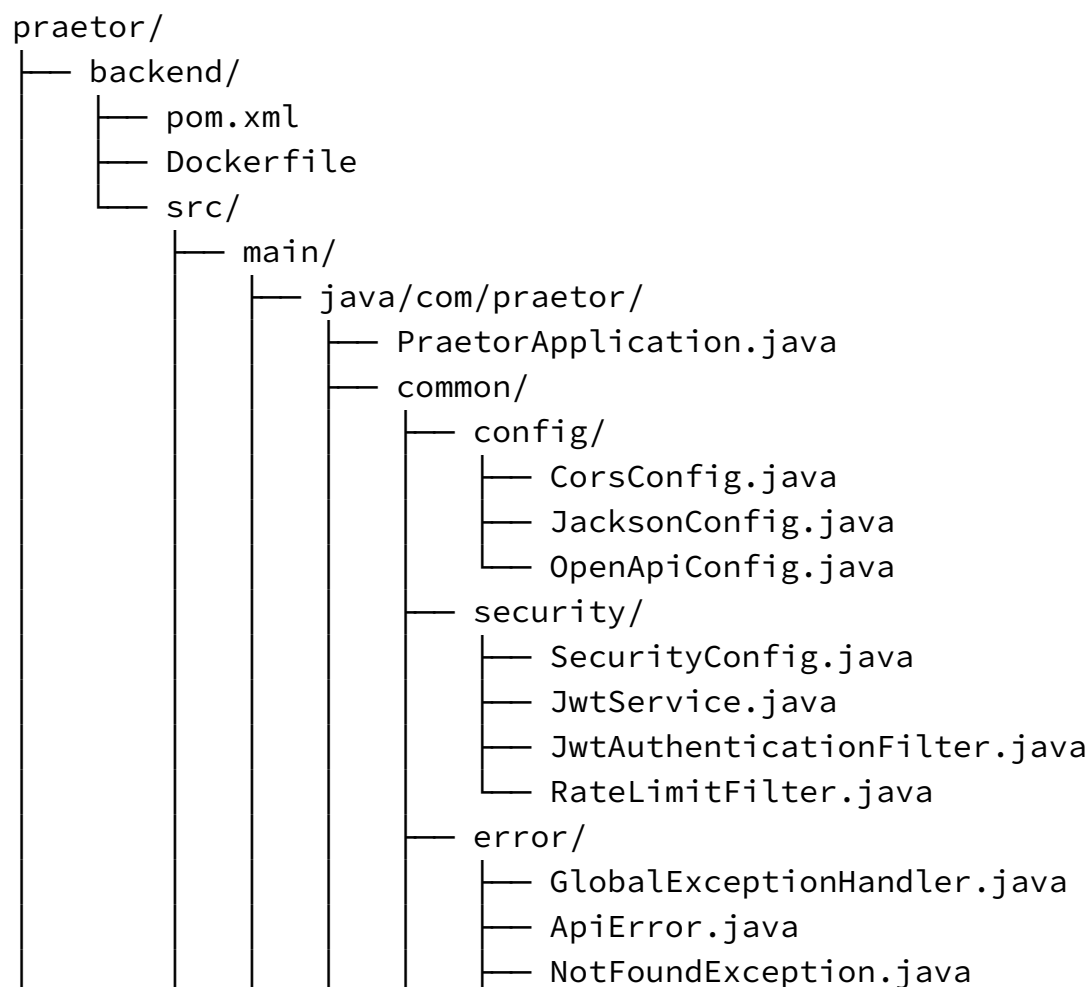
```
praetor/
├── db/
│   ├── schema.sql      # tables (migration V1)
│   └── seed.sql        # demo data – DB judgeable without any teammate
├── UI
│   ├── docs/           # api-contracts, SRS, team-plan, this file
│   └── backend/        # Spring Boot
│       └── src/main/java/com/praetor/
│           ├── problem/    # Problem, TestCase, Tag
│           ├── identity/   # User, auth, rating, stats, rate-limit filter
│           ├── submission/ # Submission, engine, sandbox runner, verdict
│           └── contest/    # Contest, standings, registration,
├── clarifications
└── ws/                 # STOMP config + topics
```



Rules

- New backend code lives under its **module package** `com.praetor.<module>`, layered entity → repository → service → controller. Do not put a controller in `common/` or an entity in `submission/` if it belongs to `problem/`.
- New Angular feature → its own folder under `features/<feature>/`. Shared building blocks go in `shared/`; app-wide singletons (guards, interceptors) in `core/`.
- One class per file. File name matches the class.

Full File planned structure:



- ForbiddenException.java
 - RateLimitException.java

- sandbox/

- DockerExecUtil.java

- identity/

- entity/

- User.java

- RatingHistory.java

- repository/

- UserRepository.java

- RatingHistoryRepository.java

- service/

- AuthService.java

- UserService.java

- RatingService.java

- StatsService.java

- controller/

- AuthController.java

- UserController.java

- RatingController.java

- LeaderboardController.java

- StatsController.java

- dto/

- RegisterRequest.java

- LoginRequest.java

- AuthResponse.java

- UserResponse.java

- RatingResponse.java

- LeaderboardEntryResponse.java

- StatsResponse.java

- problem/

- entity/

- Problem.java

- TestCase.java

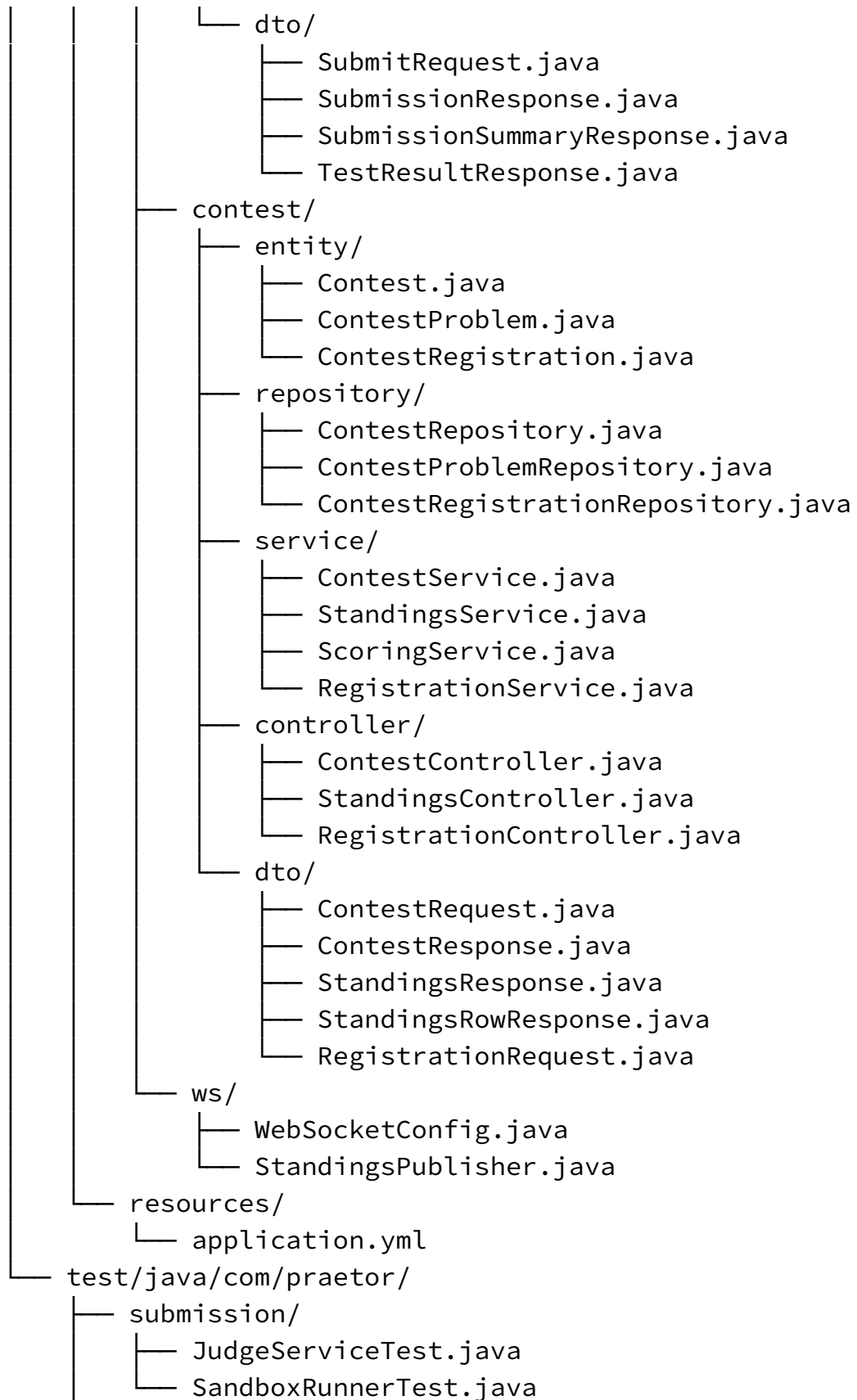
- Tag.java

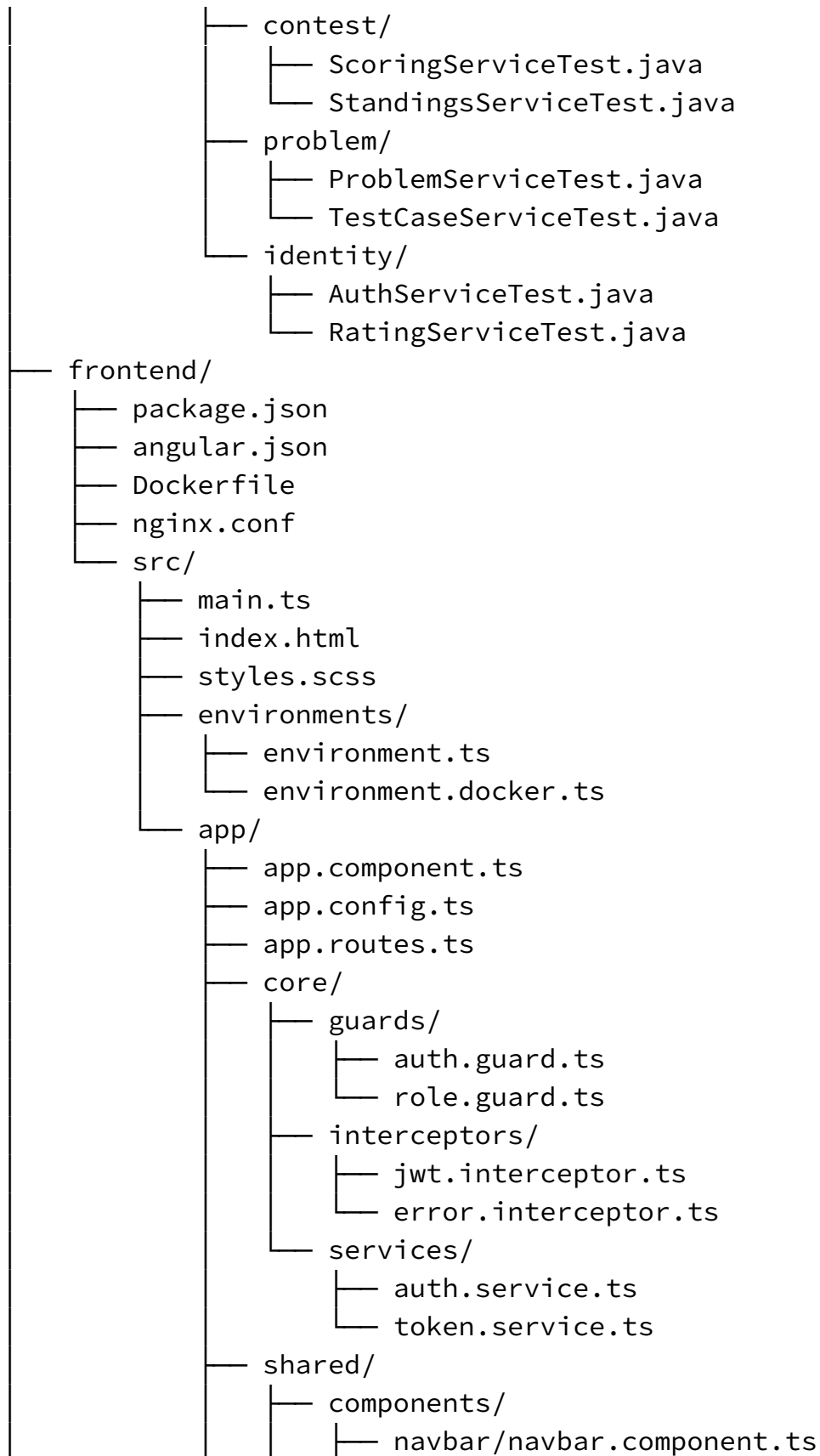
- repository/

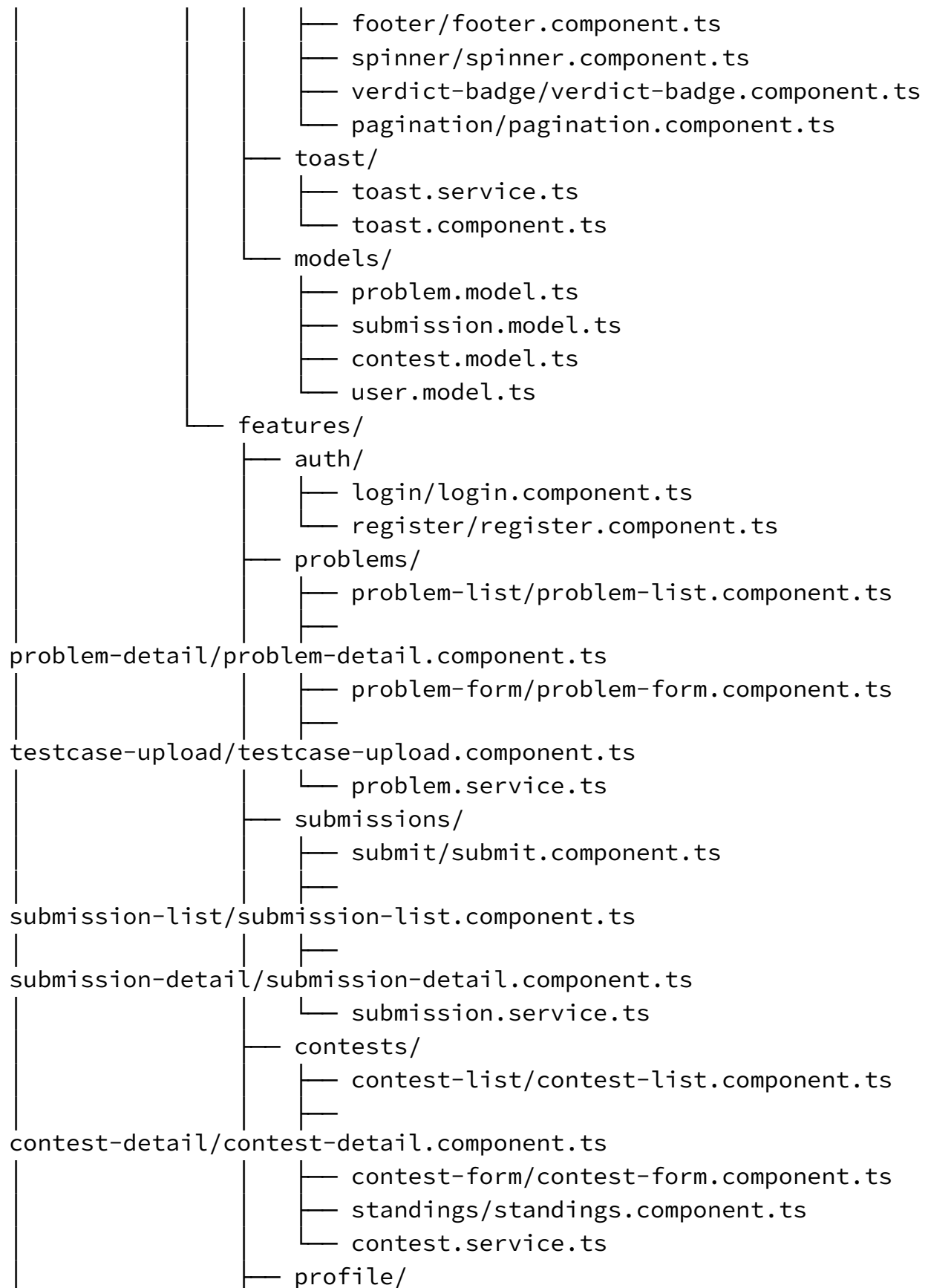
- ProblemRepository.java

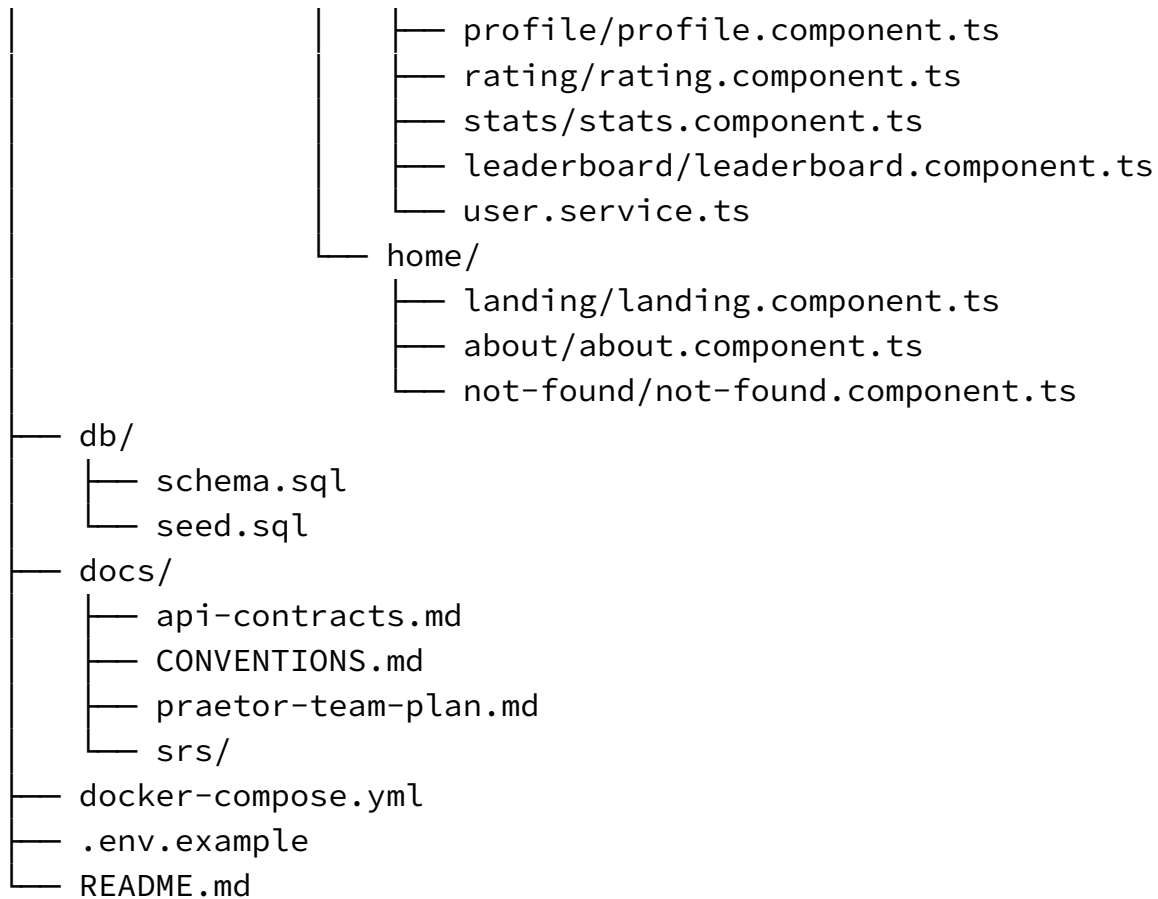
- TestCaseRepository.java

```
└─ TagRepository.java
service/
├─ ProblemService.java
├─ TestCaseService.java
├─ TagService.java
controller/
├─ ProblemController.java
├─ TestCaseController.java
├─ TagController.java
dto/
├─ ProblemRequest.java
├─ ProblemResponse.java
├─ ProblemSummaryResponse.java
├─ SampleDto.java
├─ TestCaseBulkRequest.java
submission/
├─ entity/
│   ├── Submission.java
│   └─ SubmissionResult.java
├─ repository/
│   ├── SubmissionRepository.java
│   └─ SubmissionResultRepository.java
├─ service/
│   ├── SubmissionService.java
│   ├── JudgeService.java
│   ├── SandboxRunner.java
│   ├── LanguageConfig.java
│   └─ queue/
│       ├── SubmissionQueue.java
│       └─ JudgeWorker.java
├─ checker/
│   ├── Checker.java
│   ├── ExactChecker.java
│   ├── TokenChecker.java
│   ├── FloatChecker.java
│   └─ SpecialChecker.java
controller/
└─ SubmissionController.java
```









3. Naming conventions

Layer	Rule	Good	Bad
DB table	snake_case, plural	test_cases, problem_tags	TestCase, problemTag
DB column	snake_case	time_limit_ms, created_at	timeLimitMs, createdAt
DB PK	always id (BIGSERIAL)	id	problem_id as its own PK
DB FK	<entity>_id	problem_id, created_by	probId, fk_problem
DB index	idx_<table>_<cols>	idx_sub_user_recent	index1, sub_idx
DB enum	VARCHAR + CHECK (... IN (...)), values UPPERCASE	role IN ('CODER', 'SETTER', 'ADMIN')	free-text, lowercase

Layer	Rule	Good	Bad
Java class	PascalCase + role suffix	ProblemController, ProblemService, ProblemRepository	Problems, ProblemMgr
Java field/method	camelCase	timeLimitMs, applyContestResults()	TimeLimitMS, apply_results
DTO	suffix ...Request / ...Response	SubmitRequest, SubmissionResponse	SubmissionDTO2, Payload
JSON field	camelCase	timeLimitMs, judgeMode	time_limit_ms
JSON enum value	UPPERCASE	"CPP", "AC", "QUEUED"	"cpp", "Ac"
REST path	plural nouns + {id}/{slug}	/api/problems/{slug}/testcases	/api/getProblem?id=
Slug	kebab-case, url-friendly	a-plus-b	A_Plus_B
Angular file	kebab-case + type	problem-list.component.ts	ProblemList.ts
Angular class	PascalCase + type	ProblemListComponent, SubmissionService	problemList
Git branch	type/kebab-task	feat/bulk-test case-upload	yeasir-branch, fix

Boundary reminder: the DB is snake_case, the JSON API is camelCase. JPA/Jackson maps between them — never leak snake_case into a JSON response or camelCase into a column.

4. Coding practices

4.1 MVC — what maps to what

Praetor follows **Model-View-Controller** (same mapping as praetor-team-plan.md §3):

- **Model** — JPA @Entity classes + their repositories = the data/domain, mapped to the tables in db/schema.sql. **Services live in this tier too**, as the business-logic layer sits over the Model.

- **View** — the Angular SPA (frontend/) plus the JSON the API returns. No server-side templates.
- **Controller** — Spring `@RestController`, one per module under `com.praetor.<module>`; **thin** — validate input, delegate to a Service, shape the response. Never hold business logic.

Service → Repository is the **internal layering of the Model/business tier**, not a departure from MVC — it's how the Model side stays organized. The View (Angular) talks only to Controllers over the API contract.

4.2 Backend practices (Spring)

- **Layer strictly.** Within the Model tier, Service → Repository: Services hold business logic, Repositories only do data access. **Controllers hold no business logic and run no queries** — they parse/validate input and shape responses.
- **DTOs at the boundary.** Never return a JPA `@Entity` from a controller; map to a `...Response`. Never bind request bodies straight onto entities.
- **Constructor injection** (final fields), not `@Autowired` on fields — makes testing and dependencies explicit.

```
// good
@Service
public class ProblemService {
    private final ProblemRepository problems;
    public ProblemService(ProblemRepository problems) { this.problems =
problems; }
}
```

- **No hand-built SQL.** Use JPA / parameterized queries — never string-concatenate user input into a query.
- **Validate every request DTO** (`@NotNull`, `@Size`, etc.). Reject bad input before it reaches a service.
- **One error format.** A central `@RestControllerAdvice` maps exceptions to the envelope `{ "error": "...", "status": 400 }`. Don't hand-write ad-hoc error JSON per controller.

4.3 Frontend practices (Angular)

- One **typed service per API group** (`ProblemService`, `SubmissionService`) — components call services, not `HttpClient` directly.
 - JWT is attached by the shared `HttpInterceptor` in `core/` — do **not** add the token manually per request.
 - **No business logic in components** — keep them thin; logic lives in services.
 - Type everything; avoid `any`.
-

5. API contract discipline

- The request/response shapes in `docs/api-contracts.md` **are fixed**. Implement them exactly; if a shape genuinely must change, change the contract doc first and tell the team — don't diverge silently.
 - Fixed cross-cutting rules (from the contract):
 - Base path `/api`, JSON everywhere, auth via `Authorization: Bearer <jwt>`.
 - Timestamps **ISO-8601 UTC**.
 - Errors: `{ "error": "message", "status": <code> }`.
 - Paginated lists: request `?page=0&size=20` → response `{ "content": [...], "page", "size", "totalElements" }`.
 - **Integration rules (load-bearing):**
 - The engine reads problems / test_cases **straight from the DB** (`ProblemRepository`, `TestCaseRepository`) — it does **not** call another module's controller. Broken CRUD → seed SQL still fills the tables → judging + demo survive.
 - In-process cross-module calls go through **Spring service beans** (e.g. `ContestService` calls `RatingService`) — **never HTTP** between modules in the same app.
 - **Every write endpoint validates role server-side**. Never trust the frontend to enforce SETTER/ADMIN.
-

6. Git workflow

- **Branch per task:** `type/kebab-task`. Types: `feat/`, `fix/`, `chore/`, `docs/`.
- **Never commit directly to main.** `main` stays green and demoable at all times.
- **Small PRs, reviewed by a *different* member** before merge (per Definition of Done).
- **Commit style — Conventional Commits.** Subject ≤ 50 chars, imperative mood. Body only when the “why” isn't obvious.

`feat: bulk testcase upload endpoint`

Accepts APPEND/REPLACE modes; validates ord uniqueness per problem.

Good: `fix: reject submissions after contest end` Bad: `stuff, update, WIP, asdf`

PR checklist (paste into every PR description):

- [] Branch is feat/fix/chore/docs + kebab task
 - [] Matches the shape in api-contracts.md (if it touches an endpoint)
 - [] Unit test for the service logic
 - [] Manually verified in the running app
 - [] No console / stack-trace errors
 - [] Reviewed by a different member
-

7. Docker & local dev

- **docker compose up is the contract.** It boots postgres + backend + frontend; schema.sql + seed.sql are auto-applied on a fresh DB. If your change breaks a clean docker compose up, it's not done.
 - **Sandbox runner rules (treat submitted code as hostile):**
 - **No network** in the run container.
 - Strict **CPU / memory / PID / time** limits enforced per run.
 - **Destroy the container after each run** — no reuse, no state leaking between submissions.
 - Untrusted code must never affect the host or another submission.
 - **Secrets** (DB password, JWT secret) come from **env vars**, never committed. Commit a .env.example with dummy values; keep the real .env git-ignored.
 - Pin base image versions in Dockerfiles — no bare :latest for reproducible demos.
-

8. Testing & Definition of Done

- **Backend:** JUnit for service logic; **Testcontainers** for repository/integration tests against a real Postgres (not H2).
- **Judging correctness:** keep a fixed set of known-**AC** and known-**WA** solutions per seeded problem; run them every sprint as a smoke test so a refactor can't silently break verdicts.
- **Frontend:** component tests for the submit + standings flows.

A feature is **Done** only when **all** of these hold:

1. Code merged to main via PR.
 2. Reviewed by a different member.
 3. Unit test for the service logic exists.
 4. Endpoint matches the API contract.
 5. Manually verified in the running app.
 6. No console / stack-trace errors.
-

9. Cheat sheet

- **DB** snake_case, plural tables, id PK, <entity>_id FK · **JSON** camelCase, enums UPPERCASE.
- **Java** PascalCase classes with Controller/Service/Repository suffix · constructor injection · DTOs at the boundary, never raw entities.
- **MVC** Model = JPA entities + repositories + services · View = Angular SPA + JSON · Controller = thin @RestController. Within the Model tier layer Service → Repository; no logic in controllers, no queries outside repositories.
- **API** shapes are fixed in api-contracts.md. Base /api, Bearer JWT, ISO-8601 UTC, error {error,status}, pages {content,page,size,totalElements}.
- **Integration** engine reads DB directly · cross-module = service beans, never HTTP · validate role server-side on every write.
- **Git** branch type/kebab-task · never commit to main · PR reviewed by someone else · commit type: summary ≤50 chars.
- **Docker** docker compose up must stay green · sandbox = no network + hard limits + destroy after run · secrets via env, .env git-ignored.
- **Done** = merged + reviewed + tested + contract-matched + manually verified + no errors.